

**OptiDBX Study Series · Document 2**

Viva Question Bank

45 questions a panel is likely to ask about OptiDBX, each with a grounded answer and a pointer back to the report, the scenarios, and the glossary.

Project: OptiDBX — Safe PostgreSQL Auto-Tuning

Document: 2 of the OptiDBX Study Series

Grounded in: docs/optidbx_technical_report.md

Prepared: 22 September 2026

THE OPTIDBX STUDY SERIES — READ THESE TOGETHER**1 Demo Scenarios Study Guide**

Understand every scenario the dashboard can demonstrate.

2 Viva Question Bank

Questions a panel may ask, with grounded answers.

YOU ARE HERE**3 Technical Terms Glossary**

Every term in the project, defined plainly.

5 Page-by-Page Presentation Scripts

What to say on each page of the dashboard.

HOW TO USE THIS BANK

Viva Question Bank

Nine themed sections, each question with a short defensible answer. The **Grounds** tag cites the technical report section the answer comes from; **See** points to the matching scenario (Doc 1) or glossary term (Doc 3). Numbers are quoted, never invented.

If you only revise ten — the most likely questions

1. In one sentence, what is OptiDBX? (1.1)
2. Why does reducing parallelism make queries faster? (2.2)
3. Why measure the owned workload, not `pg_stat_database`? (4.1)
4. State the net-benefit criterion (5% net, 10% regression cap). (4.5)
5. What is p95 latency and why lead with it? (4.2)
6. Why exclude warm-up from the baseline? (4.3)
7. How is a rollback verified, and what is fail-closed recovery? (5.1–5.2)
8. What keep rate did you reach, and is it cherry-picked? (6.1–6.2)
9. Name the three failure modes. (7.1–7.3)
10. What moved the system from 0% to 50%? (6.5)

Answering style. Lead with the one-line answer, then the mechanism, then a number if you have one. For any result, volunteer the caveat before you are asked — the honesty is the strongest part of the defence.

SECTION 1

Foundations & motivation

What the project is and why online tuning is hard.

1.1 In one sentence, what is OptiDBX?

A **safe online auto-tuner** for a single PostgreSQL parallelism knob (`max_parallel_workers_per_gather`) that reduces parallelism one approved step under confirmed CPU oversubscription, then **keeps the change only if warm-up-excluded, owned-workload measurements show a net benefit**, with verified rollback and fail-closed recovery.

Grounds: **Abstract** · See: Doc 1 Part A

1.2 What problem does it solve?

Online tuners must decide *while the system is live* whether a change actually helped — without destabilising the database or being fooled by noisy measurements. Keeping a harmful change degrades production; reverting a helpful one forfeits benefit and churns the system.

Grounds: §1

1.3 Offline vs online tuning — where does OptiDBX sit?

Offline tuning explores configurations on a replica or session and then installs one. **Online** tuning observes a live workload, applies a change, and must immediately judge its effect. OptiDBX is online, which is why the decision and safety layer is the hard part.

Grounds: §1

1.4 What are your three contributions?

(1) A **safe online action lifecycle** — pre-mutation journaling, verified apply, bounded observation, verified rollback, fail-closed recovery. (2) A **measurement methodology** for KEEP/ROLLBACK based on owned-workload, warm-up-excluded, client-observed metrics under a net-benefit rule. (3) An **empirical characterisation** of when parallelism reduction helps, with three documented failure modes.

Grounds: §1

1.5 Is this a new tuning algorithm?

No — that is an explicit non-goal. It is an **engineering and measurement study** focused on the online decision and safety layer, not on knob breadth or a learned multi-knob model.

Grounds: §1, §8

SECTION 2

The knob & the action

Why this parameter, and why reducing it can speed things up.

2.1 Why `max_parallel_workers_per_gather` specifically?

It caps how many parallel workers a single Gather node may use. It is a **single, reversible, bounded** action whose effect is understandable — ideal for a safety-first prototype. OptiDBX only moves it within the approved whitelist **1, 2, 4, 6, 8**, one step at a time.

Grounds: §3.3 · See: Doc 3 'per-gather parallelism'

2.2 Why does *reducing* parallelism make queries faster?

Under CPU oversubscription too many workers fight for cores; context-switching and queueing inflate the tail. Fewer workers relieve the oversubscription, so the same work completes with **lower p95 latency** while throughput stays roughly flat. In the ground-truth sweep, tail latency roughly halved (606→290 ms) as per-gather went 8→1.

Grounds: §6.4 · See: Doc 1 Scenario 1

2.3 Why not increase parallelism, or tune other knobs?

Out of the approved action set. The targeted problem is oversubscription, and the planner will not spawn more workers than a query's cost warrants — raising the shared pool to 48 left the sweep flat. Breadth is future work.

Grounds: §6.1, §9

2.4 Why restrict to a single knob at all?

Deliberate narrowing: invest complexity in **deciding correctly and acting safely** rather than exploring a large configuration space. A narrow, reversible action is what makes acting on a live database safe.

Grounds: §1

SECTION 3

Architecture & pipeline

The closed loop and the six decision stages.

3.1 Describe the closed monitoring loop.

workload → OS + DB telemetry → detection → recommendation → (display | safe apply) → bounded observation → KEEP/ROLLBACK → cooldown → monitor. There is a **strict separation** between passive monitoring and active tuning.

Grounds: §3 · See: Doc 1 Part A

3.2 What telemetry do you collect, and how often?

OS: CPU utilisation, memory pressure, disk I/O, context switches. DB: query latency, throughput, temporary-file usage, active parallel workers. Fixed **5-second** sampling. Samples with excessive timestamp skew, missing intervals, or counter resets are **rejected, not imputed**.

Grounds: §3.1 · See: Doc 3 'context switches'

3.3 How does detection decide there is a bottleneck?

A bottleneck is confirmed only after **three consecutive** problematic intervals (~15 s). Each interval must satisfy a conjunction: elevated CPU, elevated context-switch rate, high active-worker count, and elevated query latency. Thresholds are externalised and treated as calibratable, not universal.

Grounds: §3.2 · See: Doc 1 Scenario 7

3.4 Why three readings instead of acting immediately?

To debounce transient spikes. A single bad interval takes the counter to 1 of 3; the next normal reading resets it to 0. Without this, the tuner would thrash the configuration on noise.

Grounds: §3.2 · See: Doc 1 Scenario 7

3.5 Walk through the safe action lifecycle.

Journal before mutation (fsync a recovery record + persist an audit row; if either write fails, no change happens) → **verified apply** (set the value, read it back) → **bounded observation** (fixed 30 s, first interval excluded) → **decision** → **verified rollback + 30 s cooldown** → **fail-closed recovery** if a rollback cannot be verified.

Grounds: §3.3 · See: Doc 1 Scenarios 5 & 9

3.6 Which sessions can the tuner actually change?

Only an explicitly bound, **application-owned** workload session. External sessions are monitor-only, and only **one action at a time** is permitted.

Grounds: §3.3

SECTION 4

Measurement methodology (the heart)

How the KEEP/ROLLBACK question is answered — where most of the real work is.

4.1 Why measure the owned workload instead of database-wide counters?

Database-wide counters are available but unsuitable as a *decision* signal. `pg_stat_database` throughput counts all transactions — including the monitor's own commits — and is insensitive to a few large analytical queries; `pg_stat_statements` means aggregate coarsely across statement types. OptiDBX instead **times each owned query at the client** and summarises median/p95 latency and QPS — the signal the user actually feels.

Grounds: §4.1 · See: Doc 3 'owned workload'

4.2 What is p95 latency and why lead with it?

The 95th-percentile query latency — the slow tail. Parallelism reduction chiefly improves the **tail** (throughput stays flat), so p95 is the decisive signal; a mean would hide exactly the improvement that matters.

Grounds: §4.3, §6.4 · See: Doc 3 'p95 latency'

4.3 Why exclude warm-up from the baseline?

A baseline captured while the workload is still ramping (cold caches, connection warm-up) understates the baseline and **inflates the apparent improvement**. A cold baseline once made reductions look like a $\sim 2\times$ gain ($\approx 6.4 \rightarrow \approx 12$ QPS); the warm-up-excluded truth was $\approx 11 \approx 11$ QPS — no real gain.

Grounds: §4.2, §6.2 · See: Doc 3 'warm-up exclusion'

4.4 What must be true before you apply a change?

The owned workload must have been warm for a configured interval, an initial warm-up slice is excluded, and a complete warm baseline window with a **minimum of ~30 completed queries** must exist.

Grounds: §4.2, §5

4.5 State the net-benefit decision criterion precisely.

Keep when a weighted combination of throughput gain and latency improvement exceeds a **5% net threshold**. Hard guards override: never keep if tail latency regresses beyond a **10% cap**, if throughput regresses beyond its cap, or if error rate rises. OS resource degradation (CPU/memory/disk) is an **independent veto**.

Grounds: §4.3, §5 · See: Doc 3 'net-benefit rule'

4.6 Why not a latency-only or throughput-only rule?

A parallelism reduction **trades** single-query latency against aggregate behaviour, so a single-axis rule is inadequate. `latency_first` and `throughput_first` policies exist only for comparison.

Grounds: §4.3

4.7 Is every decision auditable?

Yes. Each decision records its **source** (owned vs telemetry fallback), the **measured percentage changes**, and the **policy** applied — fully reconstructable after the fact.

Grounds: §4.3

SECTION 5

Safety, rollback & recovery

What makes acting on a live database defensible.

5.1 How do you know a rollback truly restored the old value?

Verified rollback: the original value is set again and then **read back** for confirmation. Only a confirmed read-back counts as a successful rollback.

Grounds: §3.3 · See: Doc 1 Scenario 5

5.2 What is fail-closed recovery?

If a rollback cannot be verified, the system marks itself **ROLLBACK_FAILED**, retains the original sessions, and **blocks all further actions** until an operator intervenes. After a crash the on-disk journal stays authoritative and cannot be impersonated by a new connection.

Grounds: §3.3 · See: Doc 1 Scenario 9

5.3 What does 'journal before mutation' mean?

A recovery record (target, before-state, scope) is **fsync-ed** and an audit row persisted *before* any change. If either write fails, no mutation occurs — the durable journal is always ahead of the live change.

Grounds: §3.3 · See: Doc 3 'recovery journal'

5.4 How do you avoid repeatedly trying a change that never helps?

Performance-based rollbacks are recorded against a coarse workload-condition signature (parallel-worker band, CPU band). The detector then **suppresses the same reduction under comparable conditions** until a TTL elapses or conditions change materially — while still allowing reconsideration on a genuine workload shift.

Grounds: §3.4 · See: Doc 3 'rejection memory'

5.5 Why a cooldown after every action?

A 30 s cooldown follows either outcome so the system settles and the tuner does not immediately re-trigger on the same conditions.

Grounds: §3.3, §5

SECTION 6

Results & the 50% keep rate

Stated honestly — including what the number does and does not mean.

6.1 What keep rate did you achieve?

50% (3 of 6) completed actions under higher oversubscription (PostgreSQL pinned to 8 cores) — up from the system historically **never** keeping a change. At moderate oversubscription (12 cores) it was 20% (1/5), which is correctly conservative.

Grounds: §6.5 · See: Doc 1 Scenarios 1–5

6.2 Isn't 50% cherry-picked?

Be candid: **no outcome was relabelled and no run discarded**. But the regime is **constructed** — deliberately over-parallelized and CPU-constrained — realistic but not sampled from production traces, over small samples without confidence intervals. Read it as: *under the conditions the action targets, the tuner captures the available benefit*.

Grounds: §8

6.3 Why is a low keep rate not a failure?

Every completed action gave a **small real improvement**, but only those clearing the 5% net threshold were kept. Keeping marginal changes would churn the system; conservatism is the intended behaviour. Cycle 5 (worse on both axes) was correctly reverted.

Grounds: §6.5 · See: Doc 1 Scenario 5

6.4 On a healthy, well-provisioned host, what happens?

No benefit exists — the fixed-parallelism sweep spread only ~4%, within noise — so the correct action is **no action**. The workload requested only 3 workers regardless of the setting; it was memory-bandwidth bound, not CPU bound.

Grounds: §6.1 · See: Doc 1 Scenario 6

6.5 What actually moved the system from 0% to 50%?

Correct measurement and apply plumbing — owned-workload signals, warm-up exclusion, and the timestamp-refresh fix — *not* a smarter policy. The report's headline lesson is that **measurement dominates policy**.

Grounds: §7

SECTION 7

The three failure modes

The bugs that silently defeat naive online tuners — know these cold.

7.1 Failure mode 1 — the cold-start artifact?

Measuring the baseline immediately after workload start (cold caches/connections) versus a warm observation made reductions look like a **~2× throughput gain** ($\approx 6.4 \rightarrow \approx 12$ QPS). The warm-up-excluded truth was $\approx 11 \approx 11$ QPS. This motivated warm-up exclusion; any tuner comparing pre/post windows without controlling warm-up is exposed to it.

Grounds: §6.2 · See: Doc 3 'cold-start artifact'

7.2 Failure mode 2 — the stale-timestamp defect?

The auto-approval path submitted a recommendation whose timestamp was fixed at first detection while the baseline window advanced each interval. The apply-time continuity check then rejected it as discontinuous **every** interval, so a *stable* recommendation could never be applied automatically. Fix: **refresh the timestamp to the current interval at approval time**. This is consistent with the system's prior inability to ever record a KEEP.

Grounds: §6.3

7.3 Failure mode 3 — monitor starvation under saturation?

Near 100% CPU the monitoring loop shares cores with the saturating workload and cannot hold its 5 s cadence; the continuity check then intermittently rejects valid actions. There is a **constraint conflict**: detection needs high CPU, but stable monitoring needs spare CPU. Reserving CPU for the monitor (core pinning) restored cadence and enabled the keep-rate results.

Grounds: §6.6

7.4 Why do these failure modes matter beyond this project?

Two of the three **corrupt results silently** rather than failing loudly — the most dangerous class of bug for an autonomous system, because it keeps producing plausible-looking but wrong decisions.

Grounds: §7

SECTION 8

Limitations, future work & stack

Scope honesty and the engineering context.

8.1 What are the main limitations?

One parallelism knob, analytical read workloads, a single host; a **constructed** positive regime; **small samples without confidence intervals**; no comparison against other tuners or DBA heuristics; and a prototype scope (single local process, no auth/multi-node).

Grounds: §8

8.2 What is the future work?

Standard benchmarks (TPC-C/H/DS, YCSB) and real traces with confidence intervals; baselines vs default, DBA heuristics and a learned tuner; adaptive thresholds and a multi-knob extension that preserves the safety layer; portable monitor isolation (cgroups/CPU affinity); and a statistical decision procedure (e.g. sequential testing).

Grounds: §9

8.3 Is it production-ready?

No. It is a **prototype and methodology study**; the reusable contribution is the safety and measurement layer, not a deployable product.

Grounds: §8, §10

8.4 What is the technology stack?

PostgreSQL 16 on Ubuntu (WSL2) under Windows 11, 16 logical cores; a FastAPI backend; a React + Vite dashboard; dataset is pgbench scale factor 10 (~1M rows, ~150 MB).

Grounds: §5

8.5 How does the dashboard relate to the tuner?

It visualises the same six-stage pipeline. **Simulated** scenarios are illustrative walkthroughs seeded from measured runs; a **Live Session** exercises the real backend. Every report shows before/after owned-workload evidence and is labelled simulated or live.

See: Doc 1, Doc 5

SECTION 9

Curveballs

Sharper questions a panel may use to probe your understanding.

9.1 Could OptiDBX make production worse?

A bad change is caught by **bounded observation + the net-benefit rule + hard regression guards** and reverted (Scenario 5). The worst case — a rollback that cannot be verified — **fails closed** and blocks further action, so it never leaves an ambiguous, unsafe state.

Grounds: §3.3, §4.3 · See: Doc 1 Scenarios 5 & 9

9.2 What if the telemetry itself is unreliable?

Samples with excessive OS/DB timestamp skew, missing intervals, or counter resets are **rejected, not imputed**, so a decision is never made on corrupted data.

Grounds: §3.1

9.3 Why should a judge trust your KEEP decisions?

Because the **same system reverts** changes that do not help (visible rollbacks) and **does nothing** on a healthy host. The keeps are the residue of a conservative, auditable rule — not the tuner trying to justify itself.

Grounds: §6.1, §6.5

9.4 If you could keep only one sentence from this project, what is it?

The difficulty of online DB auto-tuning lies less in choosing a configuration than in **safely applying a live change and honestly measuring its effect** — measurement dominates policy.

Grounds: §10